

ARTE EN MOVIMIENTO

A Gesture-Based Interactive Drawing System to Support Creative Expression in Children

Final Project Course: Computer Vision
Academic Year: 2025 / 2026

Authors: [Bianca Cocci](#), [Francesco Faustino Greco](#)



Table of Contents

- **1- Project Overview and Motivation**
- **2- Objectives of the Proposal**
- **3- Technical Description of the Implemented System**
 - 3.1 System Architecture**
 - 3.2 Hand Tracking and Gesture Recognition**
 - 3.3 User Interface and Interaction Design**
 - 3.4 Drawing Logic and Media Management**
- **4- Sources and Technologies Used**
- **5- Conclusions and Future Work Proposals**
- **6- Desired Tools and Technologies**
- **7- Group Meeting Log**
- **8- Credits for Non-Original Materials**
- **9- First original work**

1. Project Overview and Motivation

In recent years, the field of computer vision has played an increasingly central role in **the development of natural and intuitive human–computer interaction systems**. Traditional input devices such as mouse, keyboard, or touchscreen, while effective, often represent a limitation when the goal is to enable spontaneous, expressive, and embodied interaction, especially in creative contexts.

Within this framework, ***Art in Motion*** is conceived as an interactive drawing application that allows users to create digital artwork exclusively through hand movements, captured in real time by a standard webcam.

The project is based on a **gesture-driven graphical interface**, where all interactions are performed through finger and hand movements in mid-air, without the need for physical controllers or tactile contact with screens. Drawing, tool selection, color changes, and media saving are all handled through predefined hand gestures, making the interaction continuous, fluid, and highly intuitive. The absence of traditional input devices transforms the act of drawing into a more physical and engaging experience, closer to natural body movement than to conventional digital interaction.

A core motivation behind this project is its application in the context **of children's creative development**. Children often benefit from interfaces that are immediate, playful, and exploratory, rather than rigid or text-based. A gesture-based system lowers the cognitive and technical barriers typically associated with digital drawing tools, allowing young users to focus entirely on creative expression rather than on learning how to use the interface itself. The system encourages experimentation, curiosity, and free-form creation, all of which are fundamental aspects of early creative learning.

From an educational and environmental perspective, the project also aims to promote a **sustainable approach to creative activities**. By enabling digital drawing without the use of paper or physical materials, the application offers an ecological alternative that can be used repeatedly, freely, and without material waste. This aspect is particularly relevant in educational environments, where creative activities are frequent and accessibility to materials may be limited.

In summary, the motivation of this project lies at the **intersection of computer vision, natural interaction, creativity, education, and sustainability**. The application demonstrates how vision-based technologies can be used not only for technical or industrial purposes, but also to foster human creativity in an accessible and meaningful way.

2. Objectives of the Proposal

The objectives of this project are focused on the concrete implementation of an advanced gesture-based drawing system that combines real-time hand tracking with expressive and adaptive visual feedback.

The primary implemented objectives are:

- To develop a real-time digital drawing canvas controlled by the continuous tracking of the user's hand, allowing **free-form drawing through the movement of the index finger**.
- To implement a **dynamic stroke color system** in which the color intensity and brightness vary according to the speed of the drawing gesture, enhancing visual expressiveness and reinforcing the connection between physical movement and artistic output.
- To enable **real-time stroke size control based on the distance between the thumb and the index finger**, allowing users to intuitively adjust line thickness through natural hand gestures.
- To design and implement a **mandala drawing mode**, where user input is automatically mirrored through rotational or axial symmetry, supporting creative exploration and visually rich compositions with minimal effort.
- To implement **gesture-based interaction modes that allow seamless switching between drawing, selection, and tool interaction** without relying on keyboards or mouse input.
- To provide an **intuitive graphical user interface** composed of on-screen panels and interactive zones that respond exclusively to hand gestures.
- To implement **artwork saving functionality**, allowing users to export their drawings **as image files**.
- **To implement video** recording functionality to capture the entire drawing process, enabling playback, sharing, or documentation of the creative experience.
- To ensure **real-time system performance**, maintaining smooth visual output and responsive interaction even during continuous gesture input.

Overall, the objectives of the proposal demonstrate the feasibility of applying computer vision techniques not only for interaction control, but also as a means to enrich creative processes through dynamic, intuitive, and engaging visual feedback.

3. Technical Description of the Implemented System

The system processes a live webcam stream, detects and tracks the user's hand through computer vision, and maps recognized gestures to drawing actions and UI commands. The implementation is designed for reproducibility and modularity, with a clear separation between configuration parameters, hand tracking utilities, the drawing engine, and the main interaction loop.

3.1 System Architecture

3.1.1 System Architecture and Data Flow

The application follows a real-time loop architecture:

1. Frame acquisition from a webcam using OpenCV (`cv2.VideoCapture`), with fixed resolution defined in `Config.WIDTH` and `Config.HEIGHT`.
2. Hand detection and landmark extraction using MediaPipe Hands through the `HandDetector` wrapper (`find_hands`, `find_position`).
3. Gesture interpretation based on finger states (`fingers_up`) and landmark distances (e.g., thumb–index pinch via `distance(4, 8)`).
4. Action execution inside the `PainterEngine` (e.g., drawing strokes, enabling mandala symmetry, toggling dynamic color, fill, recording).
5. Rendering: UI panels and buttons are drawn on the live frame (`draw_ui`), while the user's artwork is stored in a dedicated canvas buffer (`img_canvas`) and blended into the final output.

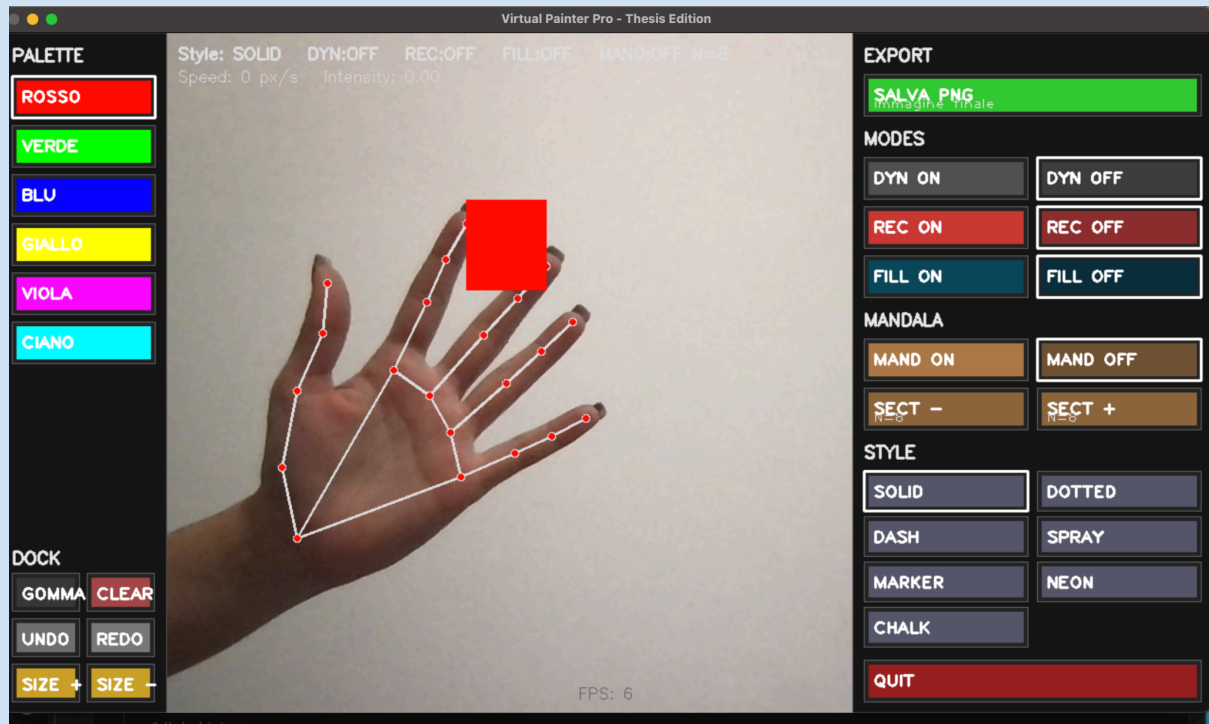
This design ensures that the drawing output remains consistent and independent from the live video background, and that each component can be tested or extended in isolation.

3.1.2 Configuration Layer

All global parameters are centralized in the `Config` class, which defines:

- Screen resolution and application naming.
- UI layout constraints (left/right panel widths, button sizes and spacing).
- Brush and eraser ranges (`BRUSH_MIN..MAX`, `ERASER_MIN..MAX`).
- Mandala defaults (supported symmetry segment counts and mirror behavior).
- Output paths for exported images and recorded videos.

3.2 Hand Tracking and Gesture Recognition



3.2.1 Hand Detection and Landmark Tracking

Hand tracking is implemented through MediaPipe Hands, wrapped by the `HandDetector` class to simplify integration:

- `find_hands(img)` runs the MediaPipe inference step and optionally draws landmarks.
- `find_position(img)` extracts pixel-space coordinates for all landmarks and stores them in `lmList`.
- `fingers_up()` computes a binary state for each finger (thumb + four fingers) using landmark geometry.
- `distance(p1, p2)` measures Euclidean distance between two landmarks, used for pinch-based controls.

The system operates with a single hand (`max_hands=1`) to maximize stability and reduce interaction ambiguity.

3.2.2 Gesture-Driven Interaction Model

The interaction model is based on three core modes, determined by finger configurations:

(A) Selection Mode (two fingers up: index + middle)

When index and middle fingers are detected as raised, the system enters selection mode. Drawing is paused, velocity estimators are reset, and the user can “touch” on-screen UI elements with the index fingertip.

In code, this corresponds to the branch:

- `if fingers[1] and fingers[2]: # selection`

During this mode, the application checks collisions between fingertip coordinates and button rectangles (`UIButton.hit(x, y)`), enabling fully touchless UI control.

(B) Drawing Mode (index finger only)

When the index finger is raised and the middle finger is down, the system enters drawing mode:

- `elif fingers[1] and not fingers[2]: # draw`

Drawing is restricted to the central canvas area, avoiding accidental painting on the UI panels. Stroke coordinates are temporally smoothed (exponential smoothing with `Config.SMOOTHING_FACTOR`) to reduce jitter.

(C) Pinch Control Mode (thumb + index only)

A dedicated pinch gesture (thumb and index raised, other fingers down) activates continuous size control. The system measures the thumb–index distance (`distance(4, 8)`), applies smoothing, and converts distance variations into stepwise updates of brush or eraser thickness. This enables intuitive “air pinch” resizing without menus.

3.3 User Interface and Interaction Design

3.3.1 Graphical User Interface Layout

The UI is fully rendered with OpenCV primitives:

- left panel: color palette + dock (eraser, clear, undo/redo, size +/-),
- right panel: mode toggles (dynamic color, recording, fill, mandala), symmetry controls, style selection grid, quit.

The interface is designed to remain functional under varying screen densities, adapting the style grid height dynamically to avoid overlap with fixed elements (QUIT button).

3.3.2 Touchless Buttons and Hit-Testing Logic

Buttons are instances of `UIButton`, which encapsulates:

- rectangle geometry,
- drawing style,
- hit testing (`hit`),
- and selection highlighting.

The interface is fully touchless: all actions are triggered by detecting collisions between fingertip coordinates and button regions during selection mode.

3.3.3 Visual Feedback and System State Indicators

Visual feedback is provided through:

- button highlighting,
- animated save confirmation,
- real-time indicators for active modes (dynamic color, recording, fill, mandala),
- overlay text reporting speed and color intensity.

These elements help the user understand the current system state without textual menus or keyboard interaction.

3.4 Drawing Logic and Media Management

3.4.1 Canvas Representation, Masking, and Stroke Rendering

The artwork is stored in:

- `img_canvas`: RGB canvas where strokes are permanently drawn.
- `paint_mask`: binary mask tracking which pixels have been painted.

This design supports:

- consistent blending into the live frame,
- basic region fill constraints,
- and correct eraser behavior.

Stroke rendering is implemented via a modular style engine (`draw_stroke`), supporting multiple artistic styles:

- SOLID, DOTTED, DASH, SPRAY, MARKER, NEON, CHALK

Each style is implemented as a separate method (e.g., `_draw_dotted`, `_draw_dash`), making the system extensible. The DASH style uses a phase accumulator (`dash_phase`) to preserve temporal continuity across segments.

3.4.2. Dynamic Color Intensity Based on Drawing Speed

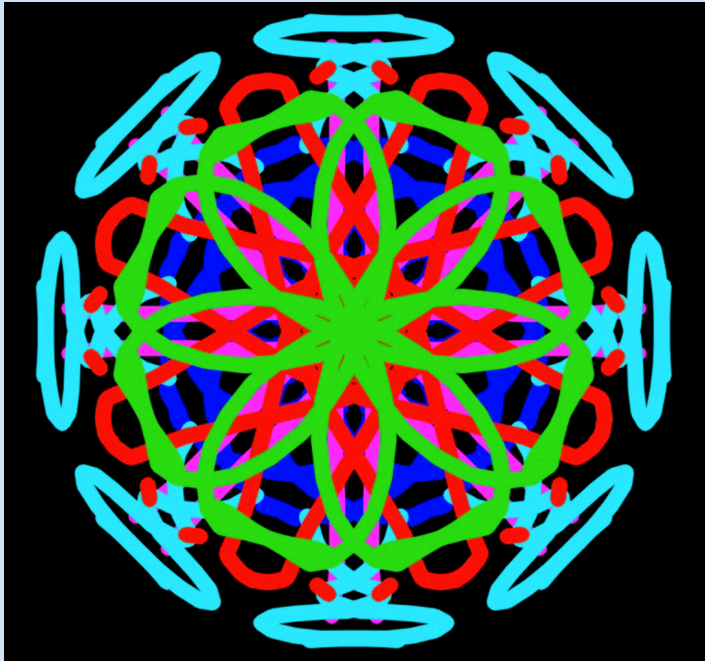
A distinctive feature of the project is the mapping between gesture speed and color intensity, enabling expressive strokes that visually reflect motion.

The speed pipeline is implemented in `PainterEngine`:

- `compute_speed(x, y)` estimates the instantaneous speed from frame-to-frame landmark displacement divided by time delta, then filters it through an exponential moving average (`speed_ema`).
- `get_dynamic_color(raw_x, raw_y)` normalizes speed into an intensity value in $[0, 1]$, using adaptive min/max estimators (`vmin_ema`, `vmax_ema`) so that the mapping remains responsive under different user motion patterns.
- `apply_dynamic_bgr(base_bgr, intensity)` converts intensity into a brightness/contrast adjustment of the current brush color.

A “stop detection” mechanism prevents undesired intensity changes when the finger is nearly still, using thresholds on both displacement and speed (`STOP_DIST_PX`, `STOP_SPEED_PX_S`) and a minimum hold time (`STOP_HOLD_SEC`). This stabilizes the experience in real usage conditions, where tiny landmark fluctuations would otherwise produce flickering.

3.4.3 Mandala Mode: Symmetry-Based Drawing



The application includes a mandala creation mode designed to support playful creativity and visually rich output with minimal effort.

When enabled (`mandala_enabled = True`), each user stroke segment is replicated into multiple symmetric segments:

- The central canvas point is treated as the symmetry center.
- The segment is rotated into `N` sectors (`MANDALA_COUNTS`), via `_rot()` and `mandala_segments()`.
- Optionally, mirrored copies are added (`MANDALA_MIRROR`).

This feature is integrated directly into the drawing pipeline through:

- `draw_segment_with_optional_mandala(...)`

Notably, mandala replication is disabled during fill operations to prevent unintended flooding interactions.

3.4.4 Fill Tool and Region Constraints

A basic fill tool is implemented using OpenCV flood fill (`cv2.floodFill`) on a temporary mask derived from the paint mask. The system only fills regions that:

- are not already painted (`paint_mask[y, x] == 0`),
- exceed a minimum area threshold (`FILL_MIN_AREA`),

to reduce accidental fills of noise-like regions. The fill action is limited to one execution per “touch” to avoid repeated flooding while the finger remains in the same area (`_fill_used_this_touch`).

3.4.5 Undo/Redo and State Management

Undo/redo is implemented via two stacks:

- `undo_stack` stores snapshots of (`img_canvas`, `paint_mask`),
- `redo_stack` stores reverted states.

Snapshots are taken at the beginning of new drawing actions (`save_state_for_undo`) and before destructive actions (clear canvas, fill). The maximum history length is limited (`MAX_HISTORY`) to avoid unbounded memory growth.

3.4.6 Media Export: Image Saving and Video Recording

The system supports two export formats:

Image export (PNG):

Triggered through the SAVE button and implemented in `save_artwork()`, which writes the current canvas to `Artworks_Gallery/` with a timestamped filename.

Video export (MP4):

A recording toggle controls an OpenCV `VideoWriter`. When recording is active, the canvas frames are appended at a fixed FPS (`VIDEO_FPS`) and saved to `Artworks_Gallery/Videos/`. Recording captures the creative process rather than the webcam background, making the output cleaner and more suitable for sharing.

3.4.7 Reproducibility Notes (Repository + README Alignment)

To ensure reproducibility, the final delivery will include:

- a complete repository containing the source code and dependencies,
- clear setup instructions (Python version, required packages),
- run commands and expected folder outputs,
- example screenshots and a short demonstration video.

In the README, the code will be documented at the same abstraction level as this section, including an explicit mapping between gestures and features (selection, draw, pinch size, dynamic color, mandala symmetry, fill, save, record), and a description of output paths and file naming conventions.

4. Sources and Technologies Used

4.1 Programming Language

The application is implemented in **Python**, chosen for its readability, rapid prototyping capabilities, and extensive ecosystem of libraries for computer vision and multimedia processing. Python enables a clear and modular code structure, which is particularly suitable for academic projects where maintainability and reproducibility are key requirements.

4.2 Computer Vision and Multimedia Libraries

- **OpenCV (cv2)**

OpenCV is used as the core multimedia framework for:

- webcam acquisition and frame handling,
- image and video rendering,
- drawing primitives (lines, shapes, text),
- video recording and export.

Its real-time performance and cross-platform support make it well suited for interactive applications.

- **MediaPipe Hands**

MediaPipe Hands, developed by Google, is used for real-time hand detection and landmark tracking. It provides 21 hand landmarks per detected hand,

enabling precise tracking of finger positions and robust gesture interpretation without the need for custom-trained models.

MediaPipe was chosen for its reliability, low latency, and ease of integration with Python and OpenCV pipelines.

- **NumPy**

NumPy is used for efficient numerical operations and array manipulation, particularly for managing image buffers, paint masks, and intermediate data structures required during drawing and fill operations.

4.3 Auxiliary Libraries and Standard Modules

- **os** and **datetime**

Used for file system management, directory creation, and timestamped naming of exported images and videos.

- **time**

Used for frame timing, speed estimation, and temporal smoothing of gesture-based parameters such as drawing velocity and dynamic color intensity.

- **math**

Used for geometric transformations, especially in the implementation of mandala symmetry through rotation and mirroring of stroke segments.

These standard Python modules contribute to the robustness of the implementation while avoiding unnecessary external dependencies.

4.4 Hardware Requirements

The system is designed to run on standard consumer hardware:

- A computer with a conventional CPU.
- A standard RGB webcam.
- No dedicated GPU or depth sensor is required.

This hardware-agnostic approach ensures that the application can be executed in typical educational or domestic environments, aligning with the project's goal of accessibility.

5. Conclusions and Future Work Proposals

5.1 Conclusions

This project has demonstrated how computer vision techniques can be effectively applied to the design of an interactive creative system that moves beyond traditional input paradigms. By relying exclusively on hand gestures captured through a standard webcam, *Art in Motion* enables a natural, continuous, and engaging drawing experience that integrates physical movement with digital expression.

Rather than focusing solely on technical performance, the project emphasizes the **role of embodied interaction in creativity**. The direct mapping between gesture dynamics and visual properties—such as stroke size controlled by pinch distance, or color intensity driven by drawing speed—reinforces the connection between motion and artistic output. This approach transforms drawing into an expressive activity where physical engagement plays a central role, rather than being reduced to discrete tool selections.

A key strength of the system lies in **its accessibility**. The absence of physical drawing tools, paper, or specialized hardware makes the application suitable for educational and domestic environments, supporting creative exploration in a **sustainable and inclusive way**. In particular, the gesture-based interface lowers the entry barrier for children, allowing them to interact with the system intuitively and focus on creative experimentation.

From a technical perspective, the project achieves real-time performance, modularity, and reproducibility through the use of open-source libraries and a clearly structured software architecture. The system successfully integrates hand tracking, gesture recognition, graphical interface management, and media export within a unified interactive loop, demonstrating a complete and practical application of computer vision concepts.

Overall, *Art in Motion* shows how vision-based interaction can be used not only as a control mechanism, but as a creative medium in itself, **opening opportunities for educational, artistic, and human-centered applications**.

5.2 Future Work Proposals

While the current implementation provides a complete and functional system, several extensions could further enhance its creative, educational, and technological potential.

One promising direction concerns **collaborative drawing between children**. Future versions of the system could support multi-hand or multi-user interaction, allowing two or more children to draw simultaneously on the same canvas. Such a feature would transform the application from an individual creative tool into a shared creative space, encouraging collaboration, cooperation, and social interaction. In educational contexts, collaborative drawing can foster communication skills, joint problem-solving, and peer learning, reinforcing creativity as a social process rather than a solitary activity.

Another important extension involves the development **of an adaptive interface tailored to different age groups**. The current system could be extended with multiple interaction modes designed for specific developmental stages, such as preschool children and primary school students. For younger users, the interface could offer simplified layouts, larger visual elements, guided mandala modes, and reduced gesture complexity. For older children, additional tools, styles, and creative freedom could be gradually introduced. Such adaptability would allow the same system to evolve with the user, supporting long-term creative engagement.

From a technological perspective, **the project could be expanded through porting to touch-based or augmented reality platforms**. A tablet version could combine gesture-based interaction with direct touch input, while an AR-based implementation could allow children to draw in three-dimensional space, blending physical movement and virtual content. The integration of depth sensors or AR frameworks would further enhance spatial interaction, opening new possibilities for immersive creative experiences.

Finally, a significant future step would be the inclusion **of user evaluation and engagement studies**. Rather than attempting to quantify creativity itself, future work could focus on observing how children interact with the system in real contexts. Qualitative observations, usage patterns, and engagement metrics—such as interaction time, diversity of tools used, and frequency of exploration—could provide valuable insights into usability, accessibility, and creative stimulation. Feedback from educators and parents could further inform design choices and support the system's integration into educational environments.

6. Desired Tools and Technologies

The development of *Art in Motion* was carried out using accessible and widely available technologies, in line with the objectives of reproducibility and ease of deployment. Nevertheless, several additional tools and technologies could further enhance the system's capabilities and open new possibilities for interaction and evaluation.

One potential improvement concerns the use of **depth-sensing devices**, such as structured-light or time-of-flight sensors. While the current implementation relies on a standard RGB camera, depth information could provide more accurate estimation of hand position in **three-dimensional space**, improving robustness under varying lighting conditions and enabling more precise gesture interpretation. Depth sensing would also support richer interaction paradigms, such as spatial drawing or layered gesture control.

Another valuable extension would involve access to hardware acceleration, particularly **GPUs**. Although the current system achieves real-time performance on standard hardware, GPU support would facilitate the integration of more advanced features, such as multi-hand interaction, machine learning-based gesture classification, or real-time augmented reality components, without compromising responsiveness.

From an interface perspective, the adoption of a **more advanced graphical user interface framework** could enhance visual quality and portability. Frameworks such as Qt or game engines could support smoother animations, richer visual feedback, and easier deployment across different platforms, while preserving the gesture-based interaction model developed in this project.

Finally, the **availability of alternative output devices**, such as large interactive displays, projectors, or tablets, could broaden the application contexts of the system. These setups would be particularly suitable for educational environments, workshops, or creative installations, further reinforcing the project's focus on collaborative and accessible creative experiences.

Overall, these tools and technologies are not prerequisites for the functionality of the current system, but rather opportunities for future enhancement. Their integration would allow *Art in Motion* to evolve beyond its current scope while maintaining the design principles established in this project.

7. Group Meeting Log

The development of this project was carried out through a combination of in-person and remote collaboration. The team worked together as much as possible in shared physical spaces such as the university facilities, the library, and co-working cafés, which facilitated direct discussion, rapid problem-solving, and iterative design.

During periods when in-person meetings were not feasible, particularly over academic breaks, coordination continued through online meetings using Google Meet. This allowed the team to maintain continuity in development, discuss design decisions, and review progress despite physical distance.

The project was managed through a shared GitHub repository, which served as the central platform for version control and code integration. Continuous communication and coordination were further supported through instant messaging, enabling quick exchanges, debugging discussions, and alignment on implementation details.

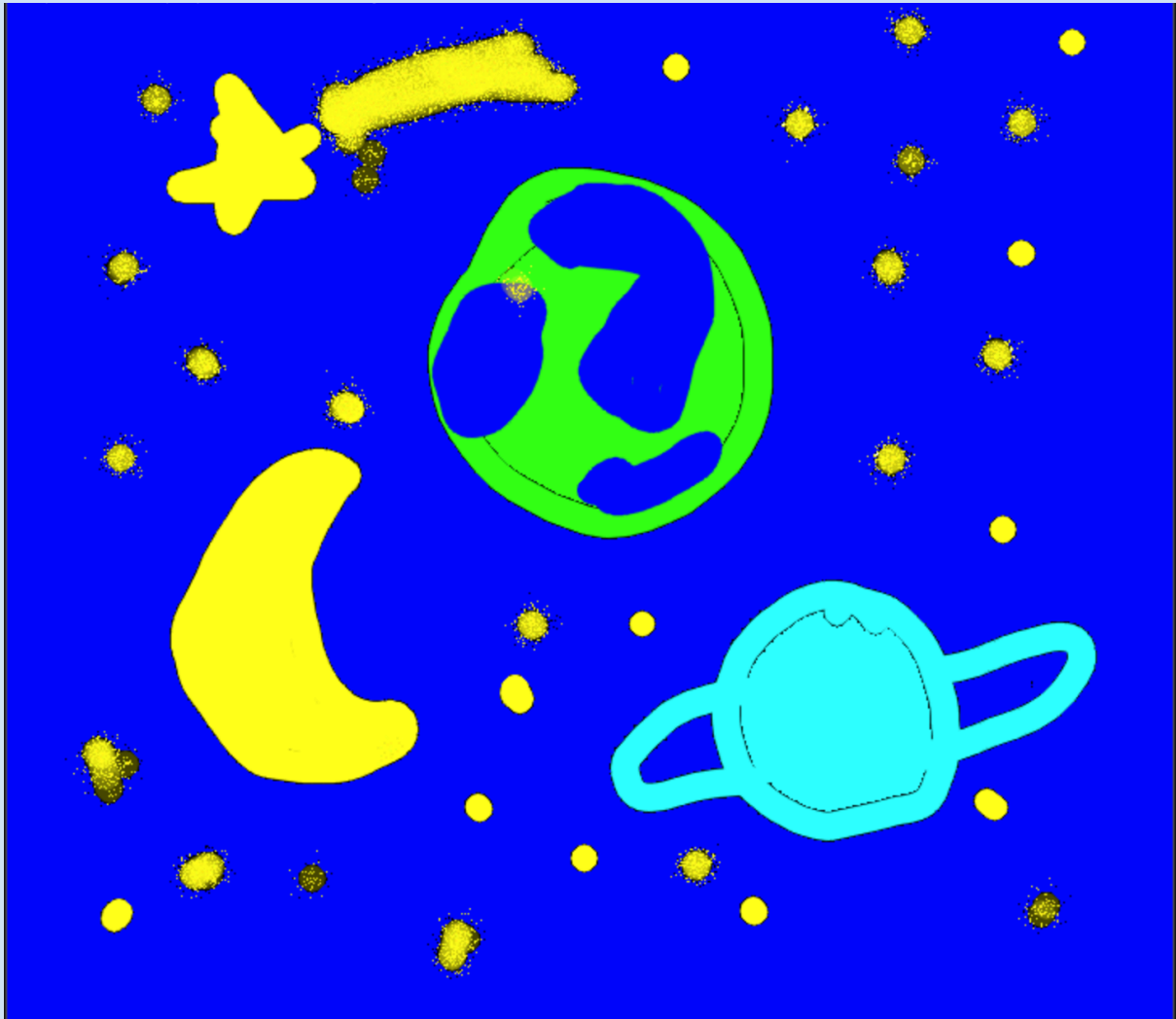
This hybrid collaboration approach ensured consistent progress throughout the project and effective teamwork across different working contexts.

8. Credits for Non-Original Materials

- Language support tools, including **ChatGPT**, were used to assist with the translation and linguistic refinement of the written documentation in English.
- The cover image of the project was created with the collaboration of **Matteo Cocci**, painter (and father of Bianca Cocci), who contributed the original artwork used for the project's cover illustration.
- **With the participation of a very young artist**, who, inspired by the magic of Christmas night, created her own interpretation of *Starry Night*. According to the artist, this version was considered more accurate and meaningful than the original. The artist reported having a great deal of fun during the creative process.

All other components of the project, including software design, implementation, and documentation structure, were developed by the project authors.

9. First original work



“Starry Night”

Created by a very, very young artist on Christmas night, 2025